PANDAS

COMPLETE CHEATSHEET

Every Command · Every Method · Interview-Ready

40+ Topics	9 Sections	200+ Commands	■30L+ Target

01	Installation & Setup
02	Data Structures — Series & DataFrame
03	Reading & Writing Data
04	Indexing, Slicing & Filtering
05	Data Cleaning & Transformation
06	GroupBy & Aggregation
07	Merging, Joining & Reshaping
08	Time Series
09	Performance & Advanced
10	Interview Quick Reference

01 · INSTALLATION & SETUP

Command	Description
`pip install pandas`	Install pandas
`pip install pandas numpy matplotlib`	Install with common deps
`conda install pandas`	Install via conda
`import pandas as pd`	Standard import alias
`import numpy as np`	Usually needed alongside
`pd.__version__`	Check pandas version
`pd.show_versions()`	Show all dependency versions

02 · DATA STRUCTURES — Series & DataFrame

Series

Command	Description
`pd.Series([1,2,3])`	Create from list
`pd.Series({'a':1,'b':2})`	Create from dict
`pd.Series([1,2], index=['x','y'])`	Custom index
`s.values`	NumPy array of values
`s.index`	Index object
`s.dtype`	Data type
`s.name`	Series name
`s.shape`	Shape tuple
`s.size`	Number of elements
`s.head(n) / s.tail(n)`	First/last n elements
`s.unique()`	Array of unique values
`s.nunique()`	Count of unique values
`s.value_counts()`	Frequency of each value
`s.sort_values()`	Sort by values
`s.sort_index()`	Sort by index

DataFrame Creation

Command	Description
`pd.DataFrame({'a':[1,2],'b':[3,4]})`	From dict of lists
`pd.DataFrame([[1,2],[3,4]], columns=['a','b'])`	From list of lists
`pd.DataFrame(np.random.rand(3,4))`	From NumPy array
`df.copy()`	Deep copy of DataFrame
`pd.DataFrame(index=range(5))`	Empty DF with index

DataFrame Inspection

Command	Description
`df.head(n) / df.tail(n)`	First/last n rows (default 5)

`df.shape`	Tuple (rows, cols)
`df.info()`	Column types, nulls, memory
`df.describe()`	Stats: count/mean/std/min/max/quartiles
`df.describe(include='all')`	Stats including object columns
`df.dtypes`	Data type of each column
`df.columns`	Column labels (Index object)
`df.index`	Row labels
`df.values`	NumPy 2D array
`df.size`	Total number of cells
`df.ndim`	Number of dimensions (always 2)
`df.memory_usage(deep=True)`	Memory per column in bytes
`df.sample(n=5)`	Random n rows
`df.sample(frac=0.1)`	Random 10% of rows

03 · READING & WRITING DATA

Reading

Command	Description
`pd.read_csv('file.csv')`	Read CSV file
`pd.read_csv('f.csv', sep='\t')`	Tab-separated file
`pd.read_csv('f.csv', header=None)`	No header row
`pd.read_csv('f.csv', index_col=0)`	First col as index
`pd.read_csv('f.csv', usecols=['a','b'])`	Load only selected cols
`pd.read_csv('f.csv', nrows=100)`	Read first 100 rows only
`pd.read_csv('f.csv', skiprows=2)`	Skip first 2 rows
`pd.read_csv('f.csv', na_values=['NA','-'])`	Custom null values
`pd.read_csv('f.csv', dtype={'col': str})`	Force dtype on load
`pd.read_csv('f.csv', chunksize=1000)`	Read in chunks (iterator)
`pd.read_excel('file.xlsx')`	Read Excel file
`pd.read_excel('f.xlsx', sheet_name='Sheet1')`	Specific sheet
`pd.read_json('file.json')`	Read JSON
`pd.read_sql(query, conn)`	Read from SQL database
`pd.read_parquet('file.parquet')`	Read Parquet (fast columnar)
`pd.read_html('url')`	Parse HTML tables from URL
`pd.read_clipboard()`	Read from clipboard

Writing

Command	Description
`df.to_csv('file.csv', index=False)`	Write CSV (no row numbers)
`df.to_csv('file.csv', sep='|')`	Custom separator
`df.to_excel('file.xlsx', index=False)`	Write Excel
`df.to_excel('f.xlsx', sheet_name='Data')`	Custom sheet name

`df.to_json('file.json')`	Write JSON
`df.to_parquet('file.parquet')`	Write Parquet (fast)
`df.to_sql('table', conn, if_exists='replace')`	Write to SQL
`df.to_clipboard()`	Copy to clipboard
`df.to_string()`	String representation
`df.to_markdown()`	Markdown table string

---

04 · INDEXING, SLICING & FILTERING

Column & Row Selection

Command	Description
`df['col']`	Select single column → Series
`df[['a','b']]`	Select multiple columns → DataFrame
`df['a':'c']  (with string index)`	Slice by label (inclusive)
`df[0:3]`	Slice first 3 rows by position

loc — Label-Based

Command	Description
`df.loc[0]`	Row with label 0
`df.loc[0:3]`	Rows label 0 to 3 (inclusive)
`df.loc[:, 'col']`	All rows, column 'col'
`df.loc[0:5, ['a','b']]`	Rows 0–5, cols a and b
`df.loc[df['age']>25]`	Filter rows where age > 25
`df.loc[condition, 'col'] = val`	Conditional assignment

iloc — Integer Position-Based

Command	Description
`df.iloc[0]`	First row
`df.iloc[-1]`	Last row
`df.iloc[0:3]`	First 3 rows (exclusive end)
`df.iloc[:, 0]`	First column
`df.iloc[0:3, 1:4]`	Rows 0-2, cols 1-3
`df.iloc[[0,2,4]]`	Rows at positions 0, 2, 4

Boolean Filtering

Command	Description
`df[df['age'] > 25]`	Single condition
`df[(df['a']>1) & (df['b']<5)]`	AND condition (use & not 'and')
`df[(df['a']==1) | (df['b']==2)]`	OR condition (use | not 'or')
`df[~df['col'].isna()]`	NOT condition (negate)
`df[df['col'].isin(['x','y'])]`	Value in list
`df[df['col'].between(10, 20)]`	Value in range (inclusive)
`df[df['col'].str.contains('abc')]`	String contains pattern

`df.query('age > 25 and salary > 50000')`	SQL-like string query
`df.query('col in @mylist')`	query() with Python variable
`df.where(df>0)`	Keep where True, else NaN
`df.mask(df>0)`	Replace where True with NaN

05 · DATA CLEANING & TRANSFORMATION

Missing Data

Command	Description
`df.isnull() / df.isna()`	Boolean mask of nulls
`df.notnull() / df.notna()`	Boolean mask of non-nulls
`df.isnull().sum()`	Count nulls per column
`df.isnull().sum() / len(df)`	% null per column
`df.dropna()`	Drop rows with any null
`df.dropna(axis=1)`	Drop columns with any null
`df.dropna(how='all')`	Drop only if ALL values null
`df.dropna(thresh=3)`	Keep rows with at least 3 non-nulls
`df.dropna(subset=['a','b'])`	Drop based on specific cols
`df.fillna(0)`	Fill nulls with scalar
`df.fillna(method='ffill')`	Forward fill (propagate last valid)
`df.fillna(method='bfill')`	Backward fill
`df.fillna(df.mean())`	Fill with column mean
`df['col'].fillna(df['col'].median())`	Fill with median
`df.interpolate()`	Linear interpolation
`df.interpolate(method='time')`	Time-based interpolation

Duplicates & Replacing

Command	Description
`df.duplicated()`	Boolean mask of duplicate rows
`df.duplicated(subset=['a','b'])`	Dups based on specific cols
`df.drop_duplicates()`	Remove duplicate rows
`df.drop_duplicates(keep='last')`	Keep last occurrence
`df.drop_duplicates(subset=['a'])`	Dedup by column 'a'
`df['col'].replace('old', 'new')`	Replace value
`df.replace({'a':'x','b':'y'})`	Replace multiple via dict
`df.replace(regex=True, ...)`	Replace with regex

Type Conversion

Command	Description
`df['col'].astype(int)`	Convert to int
`df['col'].astype(float)`	Convert to float
`df['col'].astype(str)`	Convert to string
`df['col'].astype('category')`	Convert to categorical (saves memory)

`pd.to_numeric(df['col'])`	Convert to numeric (coerce errors)
`pd.to_numeric(df['col'], errors='coerce')`	NaN on parse failure
`pd.to_datetime(df['col'])`	Convert to datetime
`pd.to_datetime(df['col'], format='%Y-%m-%d')`	Custom date format
`df.convert_dtypes()`	Auto-infer best dtypes

apply, map, transform

Command	Description
`df['col'].map(func)`	Apply func element-wise (Series)
`df['col'].map({'a':1,'b':2})`	Map via dictionary
`df['col'].apply(func)`	Apply func to Series
`df.apply(func, axis=0)`	Apply func to each column
`df.apply(func, axis=1)`	Apply func to each row
`df.apply(lambda x: x.max()-x.min())`	Range per column
`df.applymap(func)  (pandas <2.1)`	Element-wise on whole DF
`df.map(func)  (pandas 2.1+)`	Element-wise on whole DF
`df.transform(func)`	Apply and keep original shape
`df.pipe(func)`	Chain transformations cleanly

String Methods (df['col'].str.*)

Command	Description
`.str.lower() / .str.upper()`	Change case
`.str.strip() / .str.lstrip()`	Remove whitespace
`.str.replace('old','new')`	Replace substring
`.str.contains('pattern')`	Boolean: contains pattern
`.str.startswith('x')`	Boolean: starts with x
`.str.endswith('x')`	Boolean: ends with x
`.str.split('_')`	Split into list
`.str.split('_', expand=True)`	Split into columns
`.str.len()`	Length of each string
`.str.extract(r'(\d+)')`	Regex capture groups → DataFrame
`.str.findall(r'\d+')`	Find all regex matches → list
`.str.get(0)`	Get element at position 0
`.str.cat(sep=',')`	Concatenate strings
`.str.count('a')`	Count occurrences
`.str.zfill(5)`	Zero-pad to length 5

---

06 · GROUPBY & AGGREGATION

Command	Description
`df.groupby('col')`	Group by single column
`df.groupby(['a','b'])`	Group by multiple columns
`df.groupby('col').sum()`	Sum per group

`df.groupby('col').mean()`	Mean per group
`df.groupby('col').count()`	Count per group
`df.groupby('col').size()`	Size of each group (incl NaN)
`df.groupby('col').first()`	First row of each group
`df.groupby('col').last()`	Last row of each group
`df.groupby('col').min() / .max()`	Min / max per group
`df.groupby('col').std()`	Standard deviation per group
`df.groupby('col').median()`	Median per group
`df.groupby('col').agg('sum')`	Single named aggregation
`df.groupby('col').agg(['sum','mean','count'])`	Multiple aggs → MultiIndex cols
`df.groupby('col').agg({'a':'sum','b':'mean'})`	Different agg per column
`df.groupby('col').agg(total=('val','sum'))`	Named aggregation (clean output)
`df.groupby('col').transform('mean')`	Group mean, keeps original shape
`df.groupby('col').transform('rank')`	Rank within group
`df.groupby('col').filter(lambda x: len(x)>2)`	Keep groups of size > 2
`df.groupby('col').apply(custom_func)`	Custom function per group
`df.groupby('col').ngroups`	Number of groups
`df.groupby('col').get_group('val')`	Extract one group as DF

Pivot Tables & Cross-tabs

Command	Description
`pd.pivot_table(df, values='v', index='r', columns='c')`	Pivot with default mean
`pd.pivot_table(..., aggfunc='sum')`	Pivot with sum
`pd.pivot_table(..., aggfunc=['sum','count'])`	Multiple aggfuncs
`pd.pivot_table(..., margins=True)`	Add row/col totals
`pd.pivot_table(..., fill_value=0)`	Fill NaN with 0
`pd.crosstab(df['a'], df['b'])`	Frequency cross-table
`pd.crosstab(..., normalize=True)`	Proportions instead of counts
`pd.crosstab(..., margins=True)`	With row/col totals

07 · MERGING, JOINING & RESHAPING

Command	Description
`pd.merge(df1, df2, on='key')`	Inner join on column
`pd.merge(df1, df2, on='key', how='left')`	Left join
`pd.merge(df1, df2, on='key', how='right')`	Right join
`pd.merge(df1, df2, on='key', how='outer')`	Full outer join
`pd.merge(df1, df2, on=['a','b'])`	Join on multiple keys
`pd.merge(df1, df2, left_on='x', right_on='y')`	Different key names
`pd.merge(df1, df2, suffixes=('_x','_y'))`	Custom suffixes for overlapping cols
`pd.merge(df1, df2, indicator=True)`	Add _merge column (left_only etc)
`df1.join(df2, on='key')`	Join using index

`pd.merge_asof(df1, df2, on='date')`	Nearest key merge (time series)
`pd.concat([df1, df2])`	Stack rows (axis=0)
`pd.concat([df1, df2], axis=1)`	Stack columns (axis=1)
`pd.concat([df1, df2], ignore_index=True)`	Reset index after concat
`pd.concat([df1, df2], keys=['a','b'])`	Add MultiIndex keys
`df1.append(df2)`	Deprecated — use pd.concat

Reshaping

Command	Description
`df.melt(id_vars=['id'])`	Wide → long (unpivot)
`df.melt(id_vars=['id'], value_vars=['a','b'])`	Melt only selected cols
`df.pivot(index='r', columns='c', values='v')`	Long → wide (pivot)
`df.stack()`	Columns → inner row index level
`df.unstack()`	Inner index level → columns
`df.explode('col')`	Expand list cells to rows
`pd.get_dummies(df['col'])`	One-hot encode column
`pd.get_dummies(df, drop_first=True)`	Drop first to avoid multicollinearity
`pd.get_dummies(df, prefix='x')`	Custom prefix for new columns
`df.T`	Transpose (flip rows/cols)

---

08 · TIME SERIES

Command	Description
`pd.to_datetime('2024-01-01')`	Parse string to Timestamp
`pd.to_datetime(df['col'])`	Convert column to datetime
`pd.date_range('2024-01', periods=12, freq='M')`	Monthly date range
`pd.date_range(start='2024-01', end='2024-12')`	Date range between dates
`df.index = pd.DatetimeIndex(df['date'])`	Set datetime index
`df['col'].dt.year / .dt.month / .dt.day`	Extract year/month/day
`df['col'].dt.hour / .dt.minute`	Extract hour/minute
`df['col'].dt.dayofweek`	Day of week (0=Mon)
`df['col'].dt.quarter`	Quarter (1-4)
`df['col'].dt.is_month_end`	Boolean: is last day of month
`df['col'].dt.strftime('%Y-%m')`	Format as string
`df.resample('D').sum()`	Resample to daily frequency
`df.resample('W').mean()`	Resample to weekly mean
`df.resample('M').agg({'a':'sum','b':'mean'})`	Multiple aggs on resample
`df.resample('Q').last()`	Last value of each quarter
`df.rolling(window=7).mean()`	7-day rolling average
`df.rolling(window=7).std()`	7-day rolling std dev
`df.rolling('30D').sum()`	Rolling by time offset
`df.expanding().mean()`	Expanding (cumulative) mean

`df['col'].shift(1)`	Lag by 1 period
`df['col'].shift(-1)`	Lead by 1 period
`df['col'].diff(1)`	Difference from previous row
`df['col'].pct_change()`	Percentage change

---

09 · PERFORMANCE & ADVANCED

Command	Description
`df.eval('c = a + b')`	Fast column computation (avoid loop)
`pd.eval('df1 + df2')`	Fast eval across DataFrames
`df.query('a > b')`	Faster than boolean mask on large DF
`df['col'].astype('category')`	Reduce memory for low-cardinality cols
`df.memory_usage(deep=True).sum()`	Total memory in bytes
`pd.read_csv(..., chunksize=10000)`	Read large CSV in chunks
`df.pipe(step1).pipe(step2)`	Chain functions cleanly
`df.nlargest(n, 'col')`	Top n rows by column (fast)
`df.nsmallest(n, 'col')`	Bottom n rows by column (fast)
`df['col'].rank()`	Rank values (1 = smallest)
`df['col'].rank(pct=True)`	Percentile rank
`df['col'].rank(method='dense')`	Dense ranking (no gaps)
`df.clip(lower=0, upper=100)`	Cap values at min/max
`df.abs()`	Absolute values
`df.cumsum()`	Cumulative sum
`df.cumprod()`	Cumulative product
`df.cummax() / df.cummin()`	Cumulative max/min
`df.corr()`	Pairwise correlation matrix
`df.cov()`	Pairwise covariance matrix
`df.skew() / df.kurt()`	Skewness / kurtosis
`df.quantile(0.25)`	25th percentile
`df.quantile([0.25, 0.5, 0.75])`	Multiple quantiles
`pd.cut(df['col'], bins=5)`	Bin into 5 equal-width groups
`pd.qcut(df['col'], q=4)`	Bin into 4 equal-frequency quartiles

Column & Index Operations

Command	Description
`df.rename(columns={'old':'new'})`	Rename columns
`df.rename(index={0:'a'})`	Rename index labels
`df.columns = ['a','b','c']`	Set all column names at once
`df.set_index('col')`	Set column as index
`df.reset_index()`	Move index back to column
`df.reset_index(drop=True)`	Reset without adding index col
`df.drop('col', axis=1)`	Drop a column

`df.drop(['a','b'], axis=1)`	Drop multiple columns
`df.drop(0, axis=0)`	Drop a row by label
`df.insert(2, 'new_col', values)`	Insert column at position 2
`df.assign(new=df['a']+df['b'])`	Add column, returns new DF
`df.reindex(['a','b','c'])`	Reindex rows, fill NaN for missing
`df.sort_values('col')`	Sort rows by column ascending
`df.sort_values('col', ascending=False)`	Sort descending
`df.sort_values(['a','b'])`	Sort by multiple columns
`df.sort_index()`	Sort by row index

---

10 · INTERVIEW QUICK REFERENCE

Top 5 Trick Questions — Know These Cold

Q: loc vs iloc?

loc = label-based (inclusive end), iloc = position-based (exclusive end). df.loc[0:3] gives 4 rows; df.iloc[0:3] gives 3 rows.

Q: apply vs transform vs agg?

agg collapses groups (fewer rows). transform keeps original shape (good for adding group-level stats back to df). apply = most flexible, runs func on group DF.

Q: merge vs join vs concat?

merge = SQL-style join on columns. join = merge on index. concat = stack rows or cols.

Q: SettingWithCopyWarning?

Happens with chained indexing. Fix: use df.loc[row, col] = val instead of df[condition]['col'] = val. Or explicitly use .copy() after filtering.

Q: How to speed up pandas?

1) Use vectorized ops over apply/loops. 2) Use category dtype for low-cardinality strings. 3) Use eval()/query() for large DFs. 4) Read in chunks for huge CSVs. 5) Use parquet instead of CSV.

Most Tested One-Liners

Command	What it does
`df.groupby('cat').agg(total=('val','sum')).reset_index()`	Clean group totals
`df.sort_values('score').groupby('group').tail(3)`	Top 3 per group (sort first)
`df[df.duplicated(keep=False)]`	Show ALL duplicate rows
`df.isnull().sum().sort_values(ascending=False)`	Null count, worst first
`df.select_dtypes(include='number')`	Select only numeric columns
`df.select_dtypes(include='object')`	Select only string columns
`df.nunique().sort_values()`	Unique count per column
`df['col'].value_counts(normalize=True)*100`	% frequency per value
`pd.get_dummies(df, drop_first=True)`	One-hot encode all categoricals
`df.pipe(lambda d: d.assign(z=d.a+d.b)).query('z>10')`	Chain assign + filter